

Relatório da Etapa 2 — Tradutor Multilíngue em C++ com Integração Python

Gabriel Lopes Lima
2025

1 Introdução

Com o avanço das aplicações multilíngues, torna-se cada vez mais importante integrar diferentes linguagens de programação para combinar eficiência, flexibilidade e acesso a bibliotecas modernas. Este trabalho tem como objetivo desenvolver um **tradutor automático**, construído em **C++** e estendido com funcionalidades avançadas provenientes de módulos escritos em **Python**, dentre eles bibliotecas para detecção de idioma e comunicação com APIs externas.

O programa utiliza a **API C do Python**, que permite executar código Python diretamente dentro de um programa C++. Isso torna possível empregar bibliotecas como **langdetect** (detecção automática de idioma) e **requests** (comunicação com a MyMemory API). O projeto também inclui menu interativo, histórico de traduções, estrutura modular e mecanismo de despacho de ações baseado em **polimorfismo**.

Assim, o propósito central deste trabalho é estudar a integração entre C++ e Python, avaliar sua viabilidade e demonstrar seu uso em uma aplicação prática real.

2 Objetivos do Programa

O programa desenvolvido tem os seguintes objetivos:

1. **Receber um texto do usuário** e armazená-lo para operações posteriores.
2. **Detectar automaticamente o idioma** utilizando o módulo Python *langdetect*.
3. **Permitir ao usuário escolher um idioma de destino** utilizando códigos ISO (ex.: “en”, “es”, “fr”).
4. **Traduzir o texto original** para o idioma escolhido, usando a API **MyMemory** acessada via Python.
5. **Registrar todas as traduções** em um histórico manipulável dentro do próprio programa em C++.
6. **Listar todos os idiomas disponíveis**, trazendo nome completo e código ISO.

7. **Demonstrar a integração C++ ↔ Python**, incluindo criação e envio de argumentos via tuplas Python.
 8. **Aplicar princípios de programação orientada a objetos**, especialmente polimorfismo para organizar a lógica das ações do menu.
-

3 Implementação do Programa

O programa foi implementado em duas partes principais:

1. Parte C++ (main.cpp e classes de ação)

- Gerencia a interação com o usuário.
- Inicializa o interpretador Python.
- Cria objetos responsáveis por cada ação do menu.
- Concentra o histórico de traduções e o texto atual.
- Utiliza **polimorfismo** através de uma classe abstrata `AcaoMenu`, da qual derivam:
 - `A_InserirTexto`
 - `A_DetectarIdioma`
 - `A_EscolherDestino`
 - `A_Traduzir`
 - `A_ExibirHistorico`
 - `A_ListarIdiomas`

Cada opção do menu é representada por um objeto diferente que implementa o método virtual:

```
virtual void executar(string &texto,  
                      string &idioma_destino,  
                      vector<Registro> &historico,  
                      PyObject *pDetect,  
                      PyObject *pTranslate,  
                      PyObject *pListar) = 0;
```

Essa estrutura torna o programa mais modular, facilita expansão futura e melhora a organização do código.

2. Parte Python (tradutor.py)

Contém as funções chamadas pelo C++:

- `detect_language(text)`
- `translate_text(text, target)`
- `listar_idiomas()`

Essas funções utilizam bibliotecas Python para oferecer funcionalidades avançadas sem a necessidade de implementação manual em C++.

3.1 Critérios Utilizados e Justificativas

1. Uso da API C do Python

Permite acessar módulos Python e funções nativas, trazendo recursos avançados sem reimplementação em C++.

2. Classe Registro

Foi criada uma classe Registro, armazenada em Registro.h, que representa cada tradução realizada, contendo:

- texto original
- idioma de destino
- texto traduzido

3. Polimorfismo para o menu

Cada opção do menu se torna um objeto independente com comportamento próprio.

Esse critério foi escolhido por:

- facilitar manutenção do código
- permitir adicionar novas ações sem alterar o fluxo principal
- reduzir condicionais `switch/if`
- reforçar o uso de conceitos de POO

4. Uso de Vector

O histórico é armazenado em vector<Registro>, pela facilidade de manipulação dinâmica.

5. Uso de **PyTuple_Pack**

Utilizado para empacotar argumentos C++ em formato Python, permitindo chamadas seguras às funções do módulo.

6. Uso de **Py_DECREF**

Essencial para liberar referências Python e evitar vazamentos de memória.

3.2 Trechos-chave do Código

1. Inicialização do Python e inclusão do diretório atual

```
15 int main() {  
16     // ■ Inicializar Python  
17     Py_Initialize();  
18  
19     // Garantir que o diretório atual está no sys.path  
20     //sys.path é uma lista de diretórios onde o Python procura módulos  
21     PyRun_SimpleString("import sys, os");  
22     PyRun_SimpleString("sys.path.append(os.getcwd())");  
23 }
```

*Importante para garantir que o módulo **tradutor.py** possa ser encontrado e carregado.*

2. Importação do módulo Python

```
24 // ■ Importar tradutor.py  
25 PyObject *pName = PyUnicode_FromString("tradutor");  
26 PyObject *pModule = PyImport_Import(pName);  
27 Py_DECREF(pName);
```

Se o módulo não for encontrado ou contiver erros, a integração falha.

3. Chamada da função de detecção de idioma

```
PyObject* args = PyTuple_Pack(
    1, PyUnicode_FromString(texto.c_str()) //Essa função
);                                           //e coloca e

PyObject* result = PyObject_CallObject(pDetect, args);
Py_DECREF(args); // Libera a referência à tupla, permi
```

- Converte `std::string` → objeto Python (`str`)
 - Empacota em uma tupla
 - Chama a função Python `detect_language(text)`
-

4. Chamada da função de tradução

```
PyObject* args = PyTuple_Pack(
    2,
    PyUnicode_FromString(texto.c_str()), //string C++ → str Python
    PyUnicode_FromString(idioma_destino.c_str()) //string C++ → str Python
);

// Chama translate_text(texto, idioma_destino) no Python
PyObject* result = PyObject_CallObject(pTranslate, args);
```

Aqui o C++ envia 2 argumentos para Python: texto e idioma destino.

5. Trecho principal do Python para tradução

```
response = requests.get(url, params=params) #Envia a requisição para a API, Q
data = response.json() #Converte o JSON retornado para um dicionário Python

# A tradução vem aqui:
traduzido = data["responseData"]["translatedText"] #o MyMemory API devolve v
```

A API devolve um JSON que é convertido para um dicionário Python; a tradução está sempre em `responseData` → `translatedText`.

4 Casos de Uso

O programa oferece **seis funcionalidades principais**, acessadas por um menu interativo.

4.1 Exemplo de Execução Real

(1) Inserir texto

```
===== MENU TRADUTOR =====
0. Sair
1. Inserir texto
2. Detectar idioma
3. Escolher idioma de destino
4. Traduzir texto
5. Exibir histórico
6. Exibir idiomas disponíveis
Escolha: 1
Digite o texto: O cachorro está dormindo no sofá
```

(2) Detectar idioma

```
===== MENU TRADUTOR =====
0. Sair
1. Inserir texto
2. Detectar idioma
3. Escolher idioma de destino
4. Traduzir texto
5. Exibir histórico
6. Exibir idiomas disponíveis
Escolha: 2
🇧🇷 Idioma detectado: Português (pt) – confiança 1.00
```

(3) Escolher idioma de destino

```
===== MENU TRADUTOR =====
0. Sair
1. Inserir texto
2. Detectar idioma
3. Escolher idioma de destino
4. Traduzir texto
5. Exibir histórico
6. Exibir idiomas disponíveis
Escolha: 3
Digite o idioma destino (ex: en, es, fr): en
```

(4) Traduzir texto

===== MENU TRADUTOR =====

0. Sair
1. Inserir texto
2. Detectar idioma
3. Escolher idioma de destino
4. Traduzir texto
5. Exibir histórico
6. Exibir idiomas disponíveis
Escolha: 4

■ Tradução: The dog is sleeping on the couch

(5) Exibir histórico

===== HISTÓRICO DE TRADUÇÕES =====

Tradução #1:
Original: 0 cachorro está dormindo no sofá
Destino: en
Traduzido: The dog is sleeping on the couch

Tradução #2:
Original: 0 cachorro está dormindo no sofá
Destino: ja
Traduzido: 犬がソファで寝ている

Tradução #3:
Original: 0 cachorro está dormindo no sofá
Destino: tr
Traduzido: Köpek kanepede uyuyor

Tradução #4:
Original: I like to learn new things
Destino: no
Traduzido: Jeg liker å lære nye ting

Tradução #5:
Original: I like to learn new things
Destino: da
Traduzido: Jeg kan godt lide at lære nye ting

Tradução #6:
Original: I like to learn new things
Destino: de
Traduzido: Ich lerne gern dazu.

(6) Exibir idiomas disponíveis

```
===== IDIOMAS DISPONÍVEIS =====  
af – Africâner  
ar – Árabe  
bg – Búlgaro  
bn – Bengali  
ca – Catalão  
cs – Tcheco  
cy – Galês  
da – Dinamarquês  
de – Alemão  
el – Grego  
en – Inglês  
  
etc...
```

5 Conclusão

A integração entre C++ e Python mostrou-se extremamente eficiente para combinar desempenho e acesso a bibliotecas modernas. A API C do Python permitiu que o programa, além de, importar módulos, montar argumentos em tuplas, chamar funções Python e converter resultados de forma confiável, pudesse utilizar:

- Bibliotecas de detecção de idioma (*langdetect*)
- Comunicação HTTP via *requests*
- Manipulação simples de JSON
- Grande variedade de módulos Python

A principal complexidade observada ao longo do desenvolvimento foi a **integração entre C++ e Python por meio da API C do Python**. Esse processo exigiu atenção especial na criação e manipulação de **tuplas Python (PyTuple_Pack)**, que são essenciais para enviar argumentos corretamente às funções escritas em Python. A necessidade de transformar strings C++ em objetos Python (**PyUnicode_FromString**), embalá-los em tuplas e garantir que as funções fossem chamadas com a estrutura apropriada tornou essa etapa a mais trabalhosa. Apesar disso, os resultados demonstraram que a comunicação entre as linguagens é estável e eficaz, permitindo que o programa aproveite recursos avançados disponíveis no ecossistema Python sem perder o controle e a eficiência do C++.

Além disso, a implementação de um **menu polimórfico** trouxe maior organização ao programa, permitindo separar cada ação em uma classe distinta e tornando o sistema mais modular e extensível.

O programa atendeu a todos os objetivos propostos, oferecendo tradução automática, detecção de idioma, histórico de traduções e listagem de idiomas disponíveis. Com isso, conclui-se que a integração C++ ↔ Python é uma abordagem poderosa e aplicável para sistemas reais.